

Learning to Prioritize Content Refresh Opportunities Using Observed Search Signals

Abstract

Content teams working across large websites cannot manually review every page with the same level of attention, so a practical challenge is deciding which pages should be investigated first for a possible content refresh. This study uses the FlyRank ML Internship dataset to examine whether observed content and search-performance signals can help rank pages for human refresh review. A Random Forest model was trained on page-level features after excluding leakage-prone outcome fields and identifier columns, and was evaluated against the transparent ML-07 rule-based baseline using client-grouped validation. On the held-out validation set, the Random Forest improved Precision@20 from **0.40** to **0.70** and Precision@50 from **0.36** to **0.68**. These results provide directional evidence that observed page-level signals can help a FlyRank content workflow focus limited review capacity on pages showing stronger signals associated with decline. The model is a decision-support tool rather than an automatic refresh system: the results do not establish causality, do not explain Google's ranking algorithm, and do not guarantee that refreshing a recommended page will improve its future performance.

Introduction / Problem Statement

Introduction

Search performance changes over time as user behavior, search intent, competition, and content quality evolve. Because websites may contain hundreds or thousands of pages, reviewing every page manually is often impractical. Content teams therefore need a reliable way to identify which pages are most likely to benefit from review.

This project uses the **FlyRank pseudonymized content-refresh dataset** to study this prioritization problem in a realistic content workflow. In this setting, the practical challenge is deciding which pages should receive human attention first when review capacity is limited. The model therefore treats the task as a ranking problem: use observed page-level content and search-performance signals to identify pages showing stronger evidence associated with declining performance.

This project investigates whether observed content and search-performance signals can be used to prioritize pages for human content refresh review. Rather than predicting future search performance or recommending automatic content updates, the model ranks pages according to their estimated refresh opportunity using information available before the outcome period.

The objective is to support editorial decision-making by helping reviewers focus on pages that appear to have the strongest evidence of declining performance. The model produces a ranked review queue that serves as decision support, while leaving the final refresh decision to human experts.

Research Question

Can observed content and search-performance signals be used to rank pages by their priority for human refresh review?

Decision Supported

The model supports the decision of **which content pages should be reviewed first for a possible refresh**. Its output is a ranked priority list rather than an automatic recommendation to refresh content.

Why Machine Learning?

A simple rule can only evaluate a few signals at a time. Search performance, engagement, content characteristics, and page history often interact in more complex ways. Machine learning can combine these observed signals to produce a more effective ranking that helps reviewers spend their limited time on the pages most likely to require attention.

Data

Dataset

The analysis uses the **FlyRank ML Internship warehouse release**:

[*flyrank_pseudonymized_warehouse_release_v20260703*](#)

The release contains **30,000 page-level observations across 32 pseudonymized clients and 44 columns**. The unit of analysis is an individual content page.

The dataset contains observed search-performance, engagement, and content characteristics that can be used to assess whether a page may warrant human review.

Target Definition

The target is derived from the observed *trend_direction* field:

A page is assigned the declining class when *trend_direction* == "down".

The resulting target rate is **54.21%**, meaning approximately 54% of the observations belong to the declining class.

This base rate is important when interpreting the ranking results later in the paper.

Features

After exclusions, the final model uses **32 page-level features**.

These features represent observed characteristics such as:

- *Search visibility*
- *Average search position*
- *Click-through rate*
- *Page views*
- *Scroll engagement*
- *Content age*
- *Word and character count*
- *Days with impressions*
- *Days with sessions*

The features are intended to describe the page and its observed performance rather than Google's internal ranking system.

Excluded Fields

Several fields were deliberately excluded from modeling because they could leak information about the target or contain outcome-period information:

Excluded field	Reason
<i>trend_direction</i>	Directly defines the target
<i>trend_pct</i>	Contains outcome-related trend information
<i>impressions_last_30d</i>	Outcome-period performance signal
<i>clicks_last_30d</i>	Outcome-period performance signal
<i>sessions_last_30d</i>	Outcome-period performance signal
<i>impressions_prev_30d</i>	Outcome-related performance window
<i>clicks_prev_30d</i>	Outcome-related performance window
<i>sessions_prev_30d</i>	Outcome-related performance window
<i>content_id</i>	Identifier, not a predictive feature
<i>client_id</i>	Identifier used only for grouping
<i>provider_used</i>	Metadata, not a modeling feature
<i>model_used</i>	Metadata, not a modeling feature

The pseudonymous *client_id* was retained only for the grouped validation split. It was **never used as a model feature**.

Public-Safe Data Handling

The paper uses only pseudonymized and aggregate information from the internship dataset.

No client names, domains, URLs, private search queries, credentials, or raw private exports are included in the paper or its public artifacts.

The ranked recommendations use pseudonymous identifiers so that the ranking methodology can be inspected without exposing client identity.

Target Distribution

Dataset statistic	Value
Total pages	30,000
Clients	32

Features used	32
Declining class	54.21%

Methodology

Modeling Approach

The task is framed as a **decision-support classification and ranking problem**.

A Random Forest classifier is used to estimate the likelihood that a page belongs to the declining class. The resulting probability is used as a **priority score**: pages with higher scores are placed earlier in the human review queue.

The model does not directly predict whether a content refresh will succeed. It estimates which pages show signals associated with the declining class and may therefore deserve earlier review.

Target / Label

The target is defined from the observed *trend_direction* field:

down = 1 (declining), and all other observed directions = 0.

The target field itself is excluded from the model features.

Feature Set

The final feature set contains **32 page-level features**.

The features capture observed characteristics related to:

- Search demand and visibility
- Average search position
- Click-through rate
- Page views
- User engagement

- Content age
- Content length
- Frequency of observed impressions and sessions

Identifiers and outcome-related fields are excluded intentionally to reduce leakage risk.

Baseline

Before evaluating the machine-learning model, a transparent **ML-07 baseline action score** was reproduced.

The baseline assigns points for three observable conditions:

- **Stale content:** *days_since_last_update* ≥ 180
- **Search visibility:** *impressions_last_30d* ≥ 500

(The field is excluded from the Random Forest feature set because it represents outcome-period information, but it is retained for reproducing the pre-existing ML-07 baseline.)

- **Lower average position:** *avg_position* > 10

The stale-content condition receives two points, while the other two conditions receive one point each.

This provides a simple and interpretable benchmark against which the Random Forest can be compared.

Random Forest

The model is a *RandomForestClassifier* with:

- **300 trees**
- Random seed: **42**
- Balanced class weights
- Parallel training using all available CPU cores

Numeric features are median-imputed, with missing-value indicators added. Categorical features are filled using their most frequent value and one-hot encoded.

The preprocessing and model are combined in a single pipeline so that the same transformations are applied consistently during training and evaluation.

Validation Design

The dataset is divided using **GroupShuffleSplit**, with *client_id* as the grouping variable.

The split uses:

- **80% training data**
- **20% held-out test data**
- Random seed: **42**

This produced:

Split	Pages	Clients
Training	23,837	25
Test	6,163	7

There was **zero client overlap** between the training and test groups.

Grouping by client is important because allowing pages from the same client into both sets could make the evaluation overly optimistic by exposing the model to client-specific patterns during training.

Evaluation Metric

The primary metrics are **Precision@20** and **Precision@50**.

These metrics were selected because the practical use case is not to automatically classify every page. Instead, a content team has limited review capacity and needs to know which pages should be examined first.

Precision@20 measures the proportion of declining pages among the top 20 ranked recommendations.

Precision@50 measures the same quantity among the top 50 recommendations.

The baseline and Random Forest are evaluated on **exactly the same held-out test observations**.

Leakage Checks

A leakage audit was performed before modeling.

The following fields were excluded:

- *trend_direction*
- *trend_pct*
- *impressions_last_30d*
- *clicks_last_30d*
- *sessions_last_30d*
- *impressions_prev_30d*
- *clicks_prev_30d*
- *sessions_prev_30d*

content_id and *client_id* were also excluded as predictive features.

The final feature set contained **32 features**, and no suspicious feature names were identified during the leakage audit.

Modeling Assumption

The analysis assumes that observed page-level signals contain useful information for prioritizing pages for human review.

It does **not** assume that these signals establish causality or represent Google's internal ranking factors.

Results

Model vs Baseline

The Random Forest model was evaluated against the transparent ML-07 baseline action score on the **same held-out test split**. Because the intended use is to prioritize a small

number of pages for human review, Precision@20 and Precision@50 were used as the primary evaluation metrics.

Metric	ML-07 Baseline	Random Forest
Precision@20	0.40	0.70
Precision@50	0.36	0.68

The target base rate in the test set was **51.1%**. The Random Forest achieved 70% precision among its top 20 recommendations and 68% among its top 50 recommendations. This is higher than both the baseline and the overall target rate on the test set.

In the FlyRank content-refresh workflow, this means the model produced a more concentrated high-priority review queue than the simple baseline, giving human reviewers a narrower set of pages to investigate first.

At the top 20, the model therefore identified a group with a higher concentration of observed declining pages than either the baseline or the test-set base rate. At the top 50, the same pattern remained.

These results are **directional evidence** that the Random Forest provides a stronger ranking signal than the baseline on this test split. They do not establish causality and do not guarantee the same performance on future data.

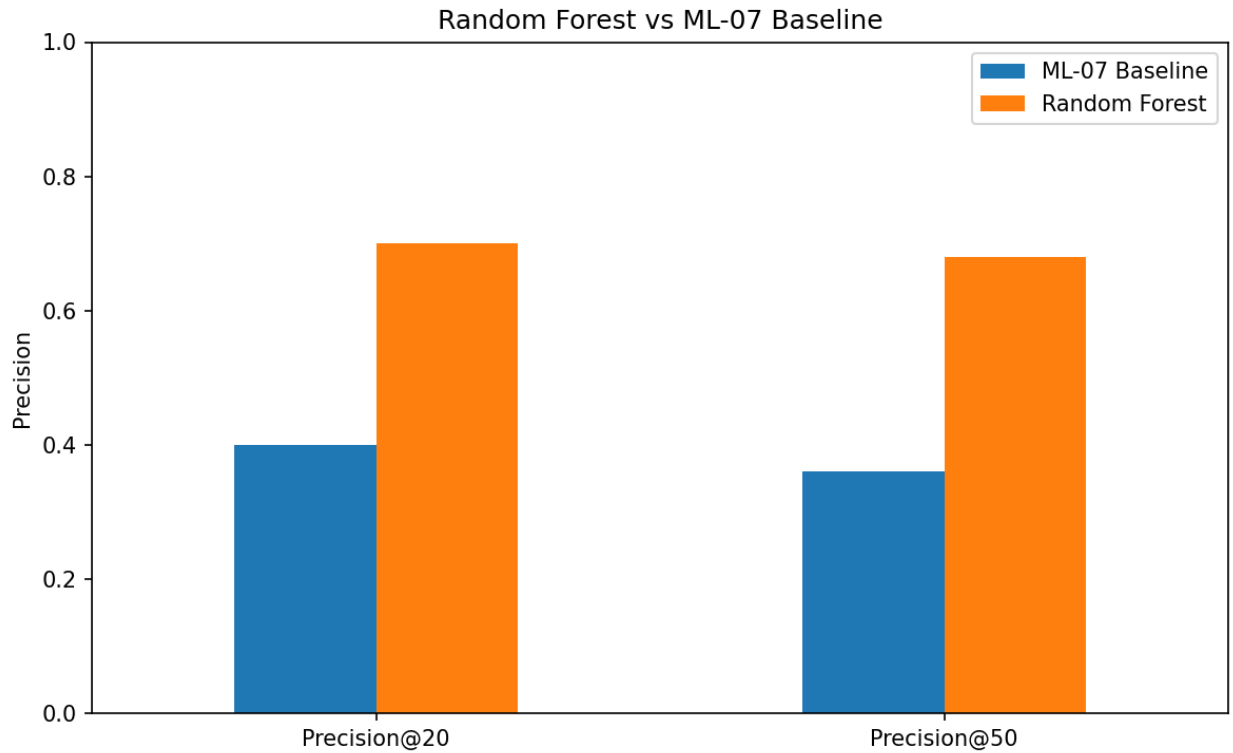


Figure 1. Precision@20 and Precision@50 for the ML-07 baseline and Random Forest on the same held-out test split.

Feature Importance

The Random Forest's most influential features were primarily related to search visibility, page position, content age, content length, and engagement.

The top features were:

Feature	Importance
<i>impressions_90d</i>	0.0887
<i>avg_position</i>	0.0878
<i>days_with_impressions</i>	0.0771
<i>content_age_days</i>	0.0627
<i>char_count</i>	0.0463
<i>word_count</i>	0.0455

<i>ctr</i>	0.0437
<i>pageviews_90d</i>	0.0411
<i>scroll_rate</i>	0.0400
<i>days_with_sessions</i>	0.0385

The strongest feature was *impressions_90d*, followed closely by *avg_position* and *days_with_impressions*. This suggests that the model relied substantially on observed search visibility and ranking characteristics when prioritizing pages.

These importances should be interpreted as **model behavior**, not as proof that any individual feature causes content decline. Feature importance also does not mean that the feature is a Google ranking factor.

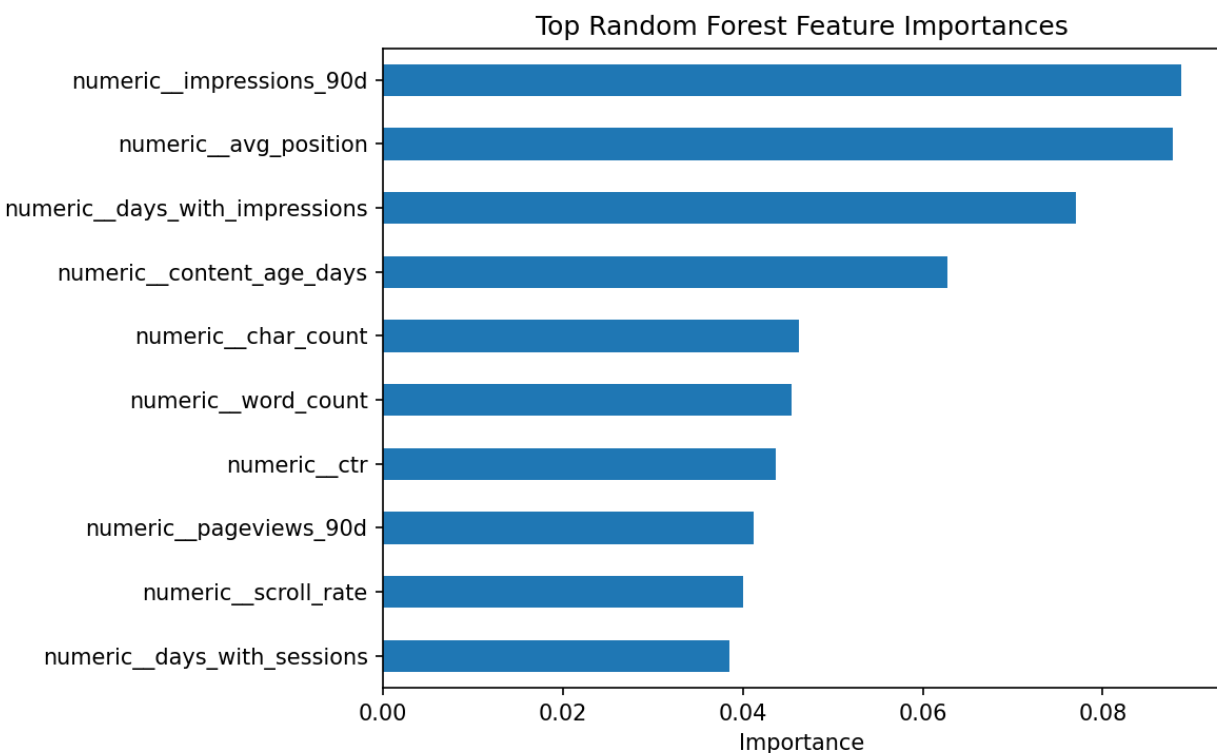


Figure 2. Top Random Forest feature importances. These values describe which available features contributed most to the model's predictions; they should not be interpreted as causal effects.

Error Analysis

The validation results also show why the model should remain a **decision-support system** rather than an automatic refresh system. Some highly ranked pages did not belong to the declining class, while some declining pages received lower scores.

For example, the ranked queue contains highly scored pages whose observed *trend_direction* was up or stable. This means that the available page-level signals are not perfectly separable and that a high model score can still produce a false positive.

These errors are important because a content editor may have additional information that is not represented in the model, such as search intent, content quality, technical problems, seasonality, business importance, or recent changes.

The model should therefore be used to **prioritize investigation**, with the final decision remaining with a human reviewer.

Limitations & Honest Framing

This model is a **decision-support tool for prioritizing pages for human review**. It does not make automatic content-refresh decisions.

The results are based on the observed signals available in the FlyRank ML Internship dataset. The analysis can identify associations between these signals and the observed declining class, but it **cannot establish that refreshing a page will cause better rankings, clicks, traffic, or engagement**.

The model also does not explain or predict Google's internal ranking algorithm. The features used in this analysis are observed content and search-performance signals, not Google's private ranking factors.

The evaluation uses a held-out client-grouped test split. This reduces the risk of the same client's pages appearing in both training and test data, but it does not guarantee that the model will perform equally well on future data or on clients with substantially different characteristics.

The model also produces false positives and false negatives. Some highly ranked pages in the review queue are not in the observed declining class, showing that the available signals do not perfectly separate the classes.

The usefulness of the model may decrease if search behavior, content mix, measurement practices, or the underlying data distribution changes. Periodic monitoring and reevaluation would therefore be necessary before relying on the ranking continuously.

Human review requirement

A high model score should be treated as a **recommendation to investigate, not an instruction to refresh**. Before taking action, a reviewer should consider:

- Search intent and content quality
- Recent changes to the page
- Technical issues
- Seasonality or external events
- Business importance
- Whether the observed decline is meaningful and recent

The final decision should always remain with a human reviewer.

Honest framing

The results are measured on one held-out test split and provide directional evidence for prioritization. They are not a causal estimate, not proof of Google's ranking factors, and not a guarantee of future performance.

Ranked Recommendations

The model converts its predictions into a **ranked review queue**. Pages with higher model scores appear earlier so that a content team can focus its limited review time on the pages with the strongest observed signals for potential refresh review.

The ranking is a **prioritization aid**, not an automatic action system.

Action Playbook

Priority	Recommended action	When to use
High	Review for refresh	High model priority and supporting evidence of potential decline
Medium	Investigate	Moderate priority or mixed signals
Low	Monitor	Lower priority or insufficient evidence for immediate action

How a FlyRank Editor Would Use It

1. Start with the highest-ranked pages.
2. Check whether the observed performance change is meaningful and recent.
3. Review the page's search intent and content quality.
4. Check for technical, seasonal, or external explanations.
5. Decide whether to **refresh, investigate further, or monitor**.
6. Record the human decision so future model performance can be evaluated.

Reason Codes

Each recommendation includes simple evidence to help explain why a page was prioritized. Examples include:

- **Older content**
- **Strong search visibility**
- **Lower average position**
- **Lower CTR**
- **Lower engagement**
- **Mixed observed signals**

These reason codes are intended to make the queue easier for a human reviewer to inspect. They are supporting evidence, not explanations of causality.

Example Ranked Queue

The model's top-ranked pages had scores as high as **0.9733**. The highest-ranked recommendations included pages with strong search visibility, lower average position, lower CTR, and lower engagement signals.

Importantly, not every highly ranked page was actually declining. Some highly ranked examples had an observed trend_direction of up or stable. This reinforces the need for human review before taking action.

priority_rank	content_id	model_score	priority	reason
1	content_98aa0aecb1d9	0.973333333	High	Strong search visibility
2	content_e988c1699454	0.973333333	High	Strong search visibility; Lower average position; Lower CTR
3	content_2ba626fea4d6	0.973333333	High	Lower CTR; Lower engagement
4	content_9ac61c04930e	0.973333333	High	Strong search visibility
5	content_7a6cb4857fe6	0.95	High	Strong search visibility; Lower CTR
6	content_500bd3907331	0.946666667	High	Strong search visibility; Lower engagement
7	content_1d0963b56227	0.946666667	High	Strong search visibility; Lower average position
8	content_cb372c0a7708	0.946666667	High	Strong search visibility; Lower average position; Lower CTR
9	content_db74537c5636	0.943333333	High	Lower average position; Lower CTR; Lower engagement
10	content_35d63627bf3e	0.943333333	High	Strong search visibility; Lower average position; Lower CTR

Reproducibility

The analysis is designed so that another person with access to the repository can inspect the notebook, reproduce the model evaluation, and regenerate the paper artifacts.

Repository

The complete project, including the capstone notebook and generated artifacts, is available in the GitHub repository:

[FlyRank AI Internship Repository \(click here\)](#)

The main notebook used for the analysis is:

[work/notebooks/capstone.ipynb](#)

Generated outputs are stored under:

[work/outputs/](#)

These include:

- *[model_comparison.csv](#)*
- *[ranked_review_queue_top50.csv](#)*
- *[top_feature_importance.csv](#)*
- *[feature_importance.png](#)*
- *[baseline_vs_random_forest.png](#)*

Reproducing the Analysis

After cloning the repository and entering the project directory, install the required dependencies:

PIP INSTALL -R REQUIREMENTS.TXT

Then open:

[work/notebooks/capstone.ipynb](#)

and run the notebook from top to bottom.

The notebook loads the released anonymized dataset, creates the target, performs the client-grouped split, trains the Random Forest, evaluates it against the ML-07 baseline, generates the ranked review queue, and exports the paper artifacts.

Test Split and Randomness

The analysis uses:

Random state: 42

The test split uses *GroupShuffleSplit* with:

Test size: 20%

Grouping variable: *client_id*

The resulting split contained:

- **23,837 training rows**
- **6,163 test rows**
- **25 training clients**
- **7 test clients**
- **0 client overlap**

The Random Forest uses **300 trees** and *class_weight*="balanced".

Reproducibility Note

The reported results should be considered reproducible when the same dataset release, code, dependencies, and random seed are used. The analysis does not claim that identical results will occur on future versions of the dataset or different environments.

Acknowledgments & Data Credit

This work was completed as part of the **FlyRank AI Machine Learning Internship**.

The analysis was built on the anonymized FlyRank ML Internship dataset and uses the released warehouse data for research and educational purposes.

Data credit: Built on the FlyRank ML Internship dataset.

[FlyRank AI](#)

I acknowledge FlyRank AI for providing the dataset and internship research environment used to develop and evaluate this work.